

1. Numpy: Illustrate the concepts multi-dimensional arrays and matrices, along with a large library of high-level mathematical functions to operate on these arrays using numpy.

Introduction

NumPy (Numerical Python) is a powerful Python library used for numerical computing. It provides support for **multi-dimensional arrays** and **matrices**, along with a large collection of **mathematical functions** to perform operations efficiently. NumPy arrays are faster and more efficient than Python lists and are widely used in data science, machine learning, and scientific computing.

Software Requirement

- Python (version 3.x)
 - NumPy Library
(Install using command: `pip install numpy`)
 - IDE/Text Editor (e.g., IDLE, VS Code, Jupyter Notebook)
-

Procedure / Steps to Execute

1. Import NumPy Library

```
import numpy as np
```

2. Create a One-Dimensional Array

```
a = np.array([1, 2, 3, 4])  
print("1D Array:", a)
```

3. Create a Multi-Dimensional Array

```
b = np.array([[1, 2, 3], [4, 5, 6]])  
print("2D Array:\n", b)
```

4. Create a Matrix

```
m = np.matrix([[1, 2], [3, 4]])  
print("Matrix:\n", m)
```

5. Perform Mathematical Operations

```
print("Addition:", a + 2)  
print("Multiplication:", a * 2)
```

6. Matrix Operations

```
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])

print("Matrix Addition:\n", m1 + m2)
print("Matrix Multiplication:\n", np.dot(m1, m2))
```

7. Use High-Level Mathematical Functions

```
print("Square Root:", np.sqrt(a))
print("Sum:", np.sum(a))
print("Mean:", np.mean(a))
print("Max:", np.max(a))
```

Output

1D Array: [1 2 3 4]

2D Array:

```
[[1 2 3]
 [4 5 6]]
```

Matrix:

```
[[1 2]
 [3 4]]
```

Addition: [3 4 5 6]

Multiplication: [2 4 6 8]

Matrix Addition:

```
[[ 6  8]
 [10 12]]
```

Matrix Multiplication:

```
[[19 22]
 [43 50]]
```

Square Root: [1. 1.41421356 1.73205081 2.]

Sum: 10

Mean: 2.5

Max: 4

6. OpenCV: Develop an online writing Whiteboard with minimal features for online classes.

Introduction

OpenCV (Open Source Computer Vision Library) is a popular Python library used for image processing and real-time computer vision applications. An online writing whiteboard allows users to draw or write on the screen using a mouse, which is useful for online classes to explain concepts visually.

Software Requirement

- Python (version 3.x)
 - OpenCV Library (`pip install opencv-python`)
 - NumPy Library (`pip install numpy`)
 - IDE/Text Editor (e.g., VS Code, Jupyter Notebook)
-

Procedure / Steps to Execute

1. Import Required Libraries

```
import cv2
import numpy as np
```

2. Initialize Variables

```
drawing = False
ix, iy = -1, -1
```

3. Create Mouse Callback Function

```
def draw(event, x, y, flags, param):
    global ix, iy, drawing

    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        ix, iy = x, y

    elif event == cv2.EVENT_MOUSEMOVE:
```

```
if drawing:
    cv2.line(img, (ix, iy), (x, y), (255, 0, 0), 3)
    ix, iy = x, y
```

```
elif event == cv2.EVENT_LBUTTONUP:
    drawing = False
```

4. Create a Blank White Canvas

```
img = np.ones((500, 800, 3), dtype=np.uint8) * 255
```

5. Set Mouse Callback

```
cv2.namedWindow("Whiteboard")
cv2.setMouseCallback("Whiteboard", draw)
```

6. Display Window and Add Controls

```
while True:
    cv2.imshow("Whiteboard", img)
    key = cv2.waitKey(1) & 0xFF

    if key == ord('c'): # Clear screen
        img[:] = 255

    elif key == 27: # Press ESC to exit
        break

cv2.destroyAllWindows()
```

Output

- A window named “**Whiteboard**” opens.
- User can **draw using mouse** by holding the left button.
- Press ‘**c**’ to clear the screen.
- Press ‘**ESC**’ to exit the application.

9. Design a perceptron classifier to classify handwritten numerical digits (0-9). Implement using Scikit or Weka.

Introduction

A Perceptron is a simple supervised machine learning algorithm used for classification tasks. It works as a linear classifier that separates data into different classes using a decision boundary. In this experiment, a perceptron classifier is used to classify handwritten digits (0–9) using the **Scikit-learn** library. The model is trained on digit image data and predicts the correct digit class.

Software Requirement

- Python (version 3.x)
 - Scikit-learn Library (`pip install scikit-learn`)
 - NumPy Library (`pip install numpy`)
 - Matplotlib (optional, for visualization)
 - IDE/Text Editor (e.g., VS Code, Jupyter Notebook)
-

Procedure / Steps to Execute

1. Install required libraries
2. Import libraries
3. Load dataset
4. Split data into training and testing
5. Train Perceptron model
6. Predict test data
7. Display accuracy and sample outputs

Program Code :

Import required libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Load dataset

```
digits = load_digits()
X = digits.data    # Features (pixel values)
y = digits.target  # Labels (digits 0-9)
```

```

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Perceptron model
model = Perceptron(max_iter=1000, eta0=0.01, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Print classification report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# Print confusion matrix
print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))

# Display some test images with predictions
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(8, 8), cmap='gray')
    plt.title(f"Pred: {y_pred[i]}")
    plt.axis('off')

plt.tight_layout()
plt.show()

```

Output

- Accuracy is displayed (typically between **85% to 95%**)
- Classification report shows precision, recall, and F1-score
- Confusion matrix is printed
- Sample handwritten digit images are displayed with predicted labels